

The Reserve

Your Website Code, Explained Line by Line

A student handout for HTML, CSS & Bootstrap
(JavaScript is covered in a separate Week 3 handout)

How to read this guide

Each part shows a **chunk of the real code** in a grey box, then explains **every line** underneath it. Read the box first, then the bullet points. Keep `index.html` and `css/styles.css` open beside this sheet so you can match what you read to the live file.

Part 0 · The Big Picture: 3 files, 3 jobs

A website is usually split into three kinds of file. Each has one job. Our project has exactly one of each:

File	Job	Real-world comparison
<code>index.html</code>	Structure — the words, images, buttons and how they are grouped.	The skeleton / bricks of a house.
<code>css/styles.css</code>	Style — the colours, fonts, spacing and the 'expensive' look.	The paint, furniture and decoration.
<code>js/script.js</code>	Behaviour — things that happen when you click (added in Week 3).	The electricity and plumbing.

The HTML file **pulls in** the other two with one line each. CSS is loaded in the head (top of the page) so the styling is ready before anything is drawn:

```
<link href="css/styles.css" rel="stylesheet">
```

- `<link>` — a self-closing tag that **links** an outside file to this page.
- `href="css/styles.css"` — the **path** to the file: the `css` folder, then `styles.css`.
- `rel="stylesheet"` — tells the browser **what kind** of file it is — a stylesheet.

We also load Bootstrap (a free, ready-made CSS library) the same way, plus the icon font and Google Fonts. Those come from the internet (a 'CDN'), which is why the page needs Wi-Fi to look right.

Part 1 · index.html, Line by Line

1.1 The page skeleton (the <head>)

Every HTML page starts with the same boilerplate. It is not visible on screen — it sets the page up.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>The Reserve | Exquisite Flavors, Timeless Heritage</title>
  ... CSS & font links ...
</head>
<body>
  ... the visible page ...
</body>
</html>
```

- `<!DOCTYPE html>` — says "this is a modern HTML5 page". Always the first line.
- `<html lang="en">` — the wrapper around everything; `lang="en"` = the page is in English.
- `<head>` — the **setup** area: title, links, settings. None of it shows on the page.
- `<meta charset="utf-8">` — lets the page show any character/emoji correctly.
- `<meta name="viewport" ...>` — makes the page fit phone screens — essential for mobile.
- `<title>...</title>` — the text shown on the browser **tab**.
- `<body>` — everything **inside** here is what visitors actually see.

1.2 The navigation bar (header)

This is the bar pinned to the top with the logo and menu links. Most class names here come from Bootstrap.

```

<header class="fixed-top">
  <nav class="navbar navbar-expand-md">
    <div class="container-fluid max-width-container px-lg-5">
      <a class="navbar-brand d-flex align-items-center gap-2" href="#home">
        <i class="bi bi-cup-hot-fill text-mahogany fs-3"></i>
        <h1 class="font-serif">The Reserve</h1>
      </a>
      <button class="navbar-toggler border-0" type="button"
        data-bs-toggle="collapse" data-bs-target="#navContent">
        <i class="bi bi-list fs-1 text-dark"></i>
      </button>
      <div class="collapse navbar-collapse justify-content-end" id="navContent">
        <ul class="navbar-nav ...">
          <li class="nav-item"><a class="nav-link active" href="#home">Home</a></li>
          ...
          <button class="btn btn-mahogany w-100"
            data-bs-toggle="modal" data-bs-target="#bookingModal">Book Now</button>
        </ul>
      </div>
    </div>
  </nav>
</header>

```

- `<header>` — a **semantic** tag meaning "top section of the page".
- `class="fixed-top"` — Bootstrap: pins the bar to the top so it stays while you scroll.
- `<nav>` — semantic tag for a block of navigation links.
- `navbar navbar-expand-md` — Bootstrap navbar that **expands** to a full row on medium screens and up; collapses to a hamburger on phones.
- `<a class="navbar-brand" href="#home">` — the logo link; `href="#home"` jumps to the section with `id="home"`.
- `<i class="bi bi-cup-hot-fill">` — an icon from Bootstrap Icons (the `bi` classes).
- `text-mahogany / fs-3` — `text-mahogany` is **our** colour class; `fs-3` is Bootstrap "font-size 3".
- `navbar-toggler` — the hamburger button shown only on small screens.
- `data-bs-toggle="collapse"` — Bootstrap magic: clicking shows/hides the menu — **no JavaScript needed from us**.
- `data-bs-target="#navContent"` — tells it **which** element to show/hide (the one with that id).
- `<ul> / <li>` — an **unordered list**; each `<li>` is one list item (one menu link).
- `nav-link active` — styles a link as part of the nav; `active` marks the current page.
- `btn btn-mahogany` — `btn` = Bootstrap button base; `btn-mahogany` = **our** custom mahogany style.
- `data-bs-target="#bookingModal"` — opens the booking pop-up (again, Bootstrap does the work).

1.3 The hero (big banner)

```

<section id="home" class="hero-section">
  <div class="hero-bg">
    
    <div class="scrim-overlay"></div>
  </div>
  <div class="hero-content">
    <span class="label-caps d-block mb-3 animate-fade-in">Established 1984</span>
    <h2 class="animate-fade-in">Exquisite Flavors, Timeless Heritage</h2>
    <button class="btn btn-mahogany px-5 py-3" ...>Book a Table</button>
    <a href="#menu" class="btn btn-outline-custom ...">Explore Menu</a>
  </div>
</section>

```

(long image links shortened to keep the code readable)

- `<section id="home">` — a labelled block of the page; the `id` lets nav links jump to it.
- `class="hero-section"` — **our** class (full-screen height) — defined in `styles.css`.
- `` — `src` = where the photo is; `alt` = text read aloud / shown if the image fails. Always write an `alt`.
- `scrim-overlay` — a dark see-through layer over the photo so white text stays readable.

- `<span class="label-caps">` — a small inline label; `label-caps` makes it tiny spaced capitals.
- `d-block` / `mb-3` — Bootstrap utilities: `d-block` = display as a block; `mb-3` = margin-bottom size 3.
- `animate-fade-in` — **our** class that fades the text in on load.
- `<button>` vs `<a>` — a `<button>` **does** something (opens the pop-up); an `<a>` **goes** somewhere (jumps to #menu).
- `px-5` `py-3` — Bootstrap padding: `px` = left/right, `py` = top/bottom, `5/3` = sizes.

1.4 The menu grid + one dish card

The menu is a Bootstrap `<b>grid</b>`: a `row` split into `col` columns. On medium screens each card takes `col-md-4` — 4 of Bootstrap's 12 columns — so three cards sit side by side. On phones they stack. Here is one card (all five are built the same way):

```
<div class="row g-4" id="menu-container">

  <div class="col-12 col-md-4 menu-col" data-category="seafood signature">
    <div class="menu-card"
      data-bs-toggle="modal" data-bs-target="#itemModal"
      data-title="Pan-Seared Scallops" data-price="$28" ...>
      <div class="menu-img-container">
        
      </div>
      <div class="d-flex justify-content-between align-items-baseline mb-1">
        <h4 class="menu-title">Pan-Seared Scallops</h4>
        <span class="menu-price">$28</span>
      </div>
      <p class="menu-desc">Succulent scallops with pea puree</p>
    </div>
  </div>

</div>
```

(long image links shortened to keep the code readable)

- `<div class="row g-4">` — starts a grid row; `g-4` adds a size-4 gap between cards.
- `col-12 col-md-4` — width: `col-12` = full width on phones; `col-md-4` = one-third on medium+ screens.
- `data-category="..."` — a **custom** attribute we invented; the Week 3 filter will read it.
- `data-title` / `data-price` — more custom data; the Week 3 pop-up reads these to fill itself in.
- `menu-img-container` — **our** class that crops every photo to the same neat shape.
- `<div class="d-flex justify-content-between">` — puts the name and price on one line, pushed to opposite ends.
- `align-items-baseline` — lines the text up neatly along the bottom of the letters.
- `<h4>` / `<span>` / `<p>` — heading, inline text, and paragraph — the dish name, price and description.

↘ Because all five cards are identical in structure, you only need to understand one. To add a sixth dish, copy one whole `col-12 col-md-4` block and change the text.

1.5 The contact form

```
<form id="contactForm" class="mt-4">
  <div class="form-floating-custom">
    <input type="text" id="contactName" placeholder=" " required>
    <label for="contactName">Your Name</label>
  </div>
  <div class="form-floating-custom">
    <input type="email" id="contactEmail" placeholder=" " required>
    <label for="contactEmail">Email Address</label>
  </div>
  <textarea id="contactMessage" rows="4" placeholder=" " required></textarea>
  <button type="submit" class="btn btn-mahogany">Send Message</button>
</form>
```

- `<form>` — groups inputs that belong together.

- `<input type="text">` — a single-line text box. The `type` changes what is allowed.
- `type="email"` — the browser checks it looks like an email address.
- `id="contactName"` — a unique name for this field, so its label (and later, JS) can find it.
- `required` — the form will not submit while this box is empty.
- `placeholder=" "` — an empty placeholder — a small trick that makes the floating label work.
- `<label for="contactName">` — `for` ties the label to the input with the matching `id`.
- `<textarea rows="4">` — a multi-line text box, 4 rows tall.
- `type="submit"` — this button **sends** the form.

🔗 Until Week 3 there is no JavaScript, so submitting reloads the page. That is normal — and it is the perfect reason to learn JavaScript next.

1.6 The footer and the closing scripts

```
<footer class="text-white py-5" style="background-color: var(--color-charcoal);">
  ... address, links, copyright ...
</footer>

<script src="https://.../bootstrap.bundle.min.js"></script>
<!-- WEEK 3: un-comment to switch on our JavaScript -->
<!-- <script src="js/script.js"></script> -->
```

(long image links shortened to keep the code readable)

- `<footer>` — semantic tag for the bottom strip of the page.
- `text-white / py-5` — Bootstrap: white text and big top/bottom padding.
- `style="background-color: var(--color-charcoal);"` — an **inline** style using one of our colour variables.
- `<script src="...bootstrap...">` — loads the Bootstrap JavaScript so the navbar and pop-ups work.
- `<!-- ... -->` — an HTML **comment**. The browser ignores it. We use it to "park" our own script until Week 3.

Part 2 · css/styles.css, Line by Line

CSS rules all follow the same shape: a **selector** (what to style) then **{ property: value; }** pairs (how to style it).

```
selector {
  property: value;
}
```

- **selector** — picks elements: a tag (`body`), a class (`.btn-mahogany`), or an id (`#home`).
- **property: value;** — one styling instruction, e.g. `color: red;`. Always ends with a semicolon.

2.1 Brand variables (:root)

Variables let us name a colour once and reuse it everywhere. Change it here and the whole site updates.

```
:root {
  --color-cream:    #fdf8f8;
  --color-charcoal: #1c1b1b;
  --color-mahogany: #894e47;
  --font-serif: 'EB Garamond', Georgia, serif;
}
```

- `:root` — means "the whole document"; variables defined here work everywhere.
- `--color-mahogany: #894e47;` — creates a variable. `#894e47` is a colour written in hex (red-green-blue).
- `var(--color-mahogany)` — how you **use** the variable later, e.g. `color: var(--color-mahogany);`.
- `--font-serif: ...;` — a font variable. The list is fall-backs: if the first font is missing, try the next.

2.2 The body (page defaults)

```
body {  
  background-color: var(--color-cream);  
  color: var(--color-charcoal);  
  font-family: var(--font-sans);  
  font-size: 16px;  
  line-height: 1.6;  
}
```

- `background-color` — the page background — our cream colour.
- `color` — the default **text** colour for the whole page.
- `font-family` — which font the text uses.
- `font-size: 16px;` — base text size (`px` = pixels).
- `line-height: 1.6;` — space between lines — 1.6x the font size makes text easy to read.

2.3 A custom button

```
.btn-mahogany {  
  background-color: var(--color-mahogany);  
  color: #ffffff;  
  border: none;  
  border-radius: 4px;  
  padding: 0.75rem 2rem;  
  text-transform: uppercase;  
  transition: all 0.3s ease;  
}  
.btn-mahogany:hover {  
  background-color: var(--color-mahogany-hover);  
  transform: translateY(-2px);  
}
```

- `.btn-mahogany` — the dot means "elements with class `btn-mahogany`".
- `border: none;` — removes the default button outline.
- `border-radius: 4px;` — rounds the corners slightly.
- `padding: 0.75rem 2rem;` — inner space: top/bottom 0.75rem, left/right 2rem (`rem` scales with font size).
- `text-transform: uppercase;` — shows the text in CAPITALS.
- `transition: all 0.3s ease;` — animates **any** change smoothly over 0.3 seconds.
- `:hover` — a special selector: these rules apply only when the mouse is over the button.
- `transform: translateY(-2px);` — nudges the button up 2px on hover — a subtle "lift".

2.4 The full-screen hero

```
.hero-section {  
  position: relative;  
  height: 100vh;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

- `position: relative;` — lets us place the photo and dark layer precisely **inside** this section.
- `height: 100vh;` — `vh` = viewport height; `100vh` = exactly one full screen tall.
- `display: flex;` — turns on Flexbox — a powerful way to centre and arrange children.
- `align-items: center;` — centres the content **vertically**.
- `justify-content: center;` — centres the content **horizontally**.

2.5 The floating label trick

This is the cleverest CSS in the file. The label starts inside the input, then floats up when you click in or type.

```
.form-floating-custom input:focus ~ label,
.form-floating-custom input:not(:placeholder-shown) ~ label {
  top: -1.25rem;
  font-size: 10px;
  color: var(--color-mahogany);
}
```

- `:focus` — applies while the input is selected (clicked into).
- `:not(:placeholder-shown)` — applies when the input is **not** empty — i.e. you have typed something.
- `~ label` — the "sibling" combinator: find the `label` that comes after the input.
- `top: -1.25rem;` — moves the label **up** above the box.
- `font-size: 10px;` — shrinks it as it floats up.

2.6 A keyframe animation

```
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}
.animate-fade-in {
  animation: fadeIn 1s ... forwards;
}
```

- `@keyframes fadeIn` — defines an animation named `fadeIn`.
- `from { ... }` — the starting look: invisible (`opacity: 0`) and nudged down 20px.
- `to { ... }` — the ending look: fully visible and in its normal position.
- `animation: fadeIn 1s forwards;` — plays `fadeIn` over 1 second and **keeps** the final look (`forwards`).

Part 3 · Bootstrap Class Cheat-Sheet

These are the Bootstrap classes used in this project. Hand this table out — it answers 90% of "what does that class mean?" questions.

Layout & grid

Class	What it does
<code>container</code> / <code>container-fluid</code>	Centres content with sensible margins (fluid = full width).
<code>row</code>	Starts a horizontal row of columns.
<code>col-12</code>	Take all 12 columns (full width).
<code>col-md-4</code>	Take 4 of 12 columns on medium screens and up (one-third).
<code>g-4</code>	Add a gap (size 4) between grid items.
<code>order-1</code> / <code>order-2</code>	Re-order items on screen without changing the HTML.

Flexbox & alignment

Class	What it does
<code>d-flex</code>	Turn on Flexbox for this element.
<code>justify-content-between</code>	Push children to opposite ends.
<code>justify-content-center</code>	Centre children horizontally.
<code>align-items-center</code>	Centre children vertically.

Class	What it does
gap-2 / gap-3	Space between flex children.
flex-column	Stack children top-to-bottom instead of side-by-side.

Spacing (margins & padding)

Pattern	What it does
m = margin, p = padding	Outer space (m) vs inner space (p).
mb-3	m argin- b ottom, size 3 (t=top, b=bottom, s=start, e=end).
px-5	p adding left+right (x), size 5 (y = top+bottom).
py-3 / mt-2	padding top+bottom; margin-top size 2.
Sizes 0-5	0 = none, 5 = largest.

Text, colour & components

Class	What it does
text-center / text-md-start	Centre text (and left-align on medium+).
text-white / text-muted	White text / soft grey text.
fw-light / fst-italic	Light font weight / italic.
fs-1 ... fs-3	Font sizes (1 = biggest).
btn / btn-link	Button base / a button that looks like a link.
modal / fade	Pop-up dialog box / fade it in and out.
d-none	Hide the element completely.
shadow-lg / rounded	Large drop shadow / rounded corners.

Part 4 · Words You Will Keep Hearing

Term	Plain-English meaning
Tag	A building block in angle brackets, e.g. Usually opens and closes.
Element	A tag plus everything inside it.
Attribute	Extra info inside a tag, written name="value" (e.g. href, src, class).
Class	A label you put on elements so CSS can style many at once (class="...").
id	A unique label — only one element per page should have a given id.
Selector	The part of a CSS rule that chooses what to style (tag, .class, or #id).
Property / value	One CSS instruction, e.g. color: red;.
Variable	A saved value (a colour or font) you can reuse with var(--name).
Semantic tag	A tag whose name describes its meaning: header, nav, main, section, footer.
Bootstrap	A free library of ready-made CSS classes (and some JS) so you don't start from zero.
CDN	A web address that serves shared files like Bootstrap — needs internet.

Term	Plain-English meaning
Comment	Notes for humans the browser ignores: <!-- in HTML --> and / in CSS /.

You've got this. Read a chunk, find it in the live file, change one value, and refresh the browser. That loop — read, change, refresh — is how every web developer learns.